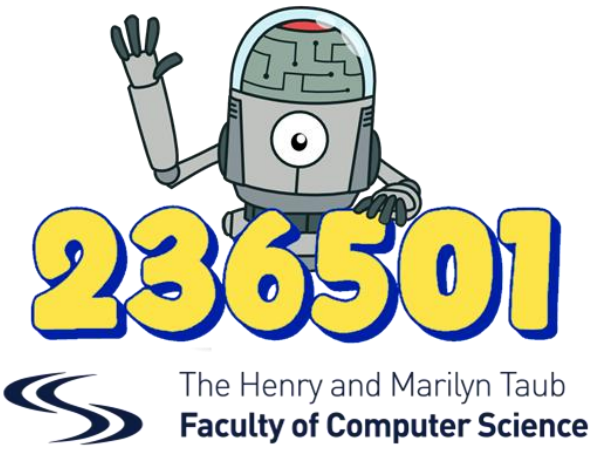
מבוא לבינה מלאכותית – 236501

חורף תשפ״ו, 2026

תרגיל בית 1

<u>מרחבי חיפוש, אלגוריתמי חיפוש מיודע ולא מיודע, חיפוש לוקאלי</u>



מטרות התרגיל:

- נתמודד עם בעיות פרקטיות ותיאורטיות של חיפוש במרחבי מצבים.
- נתרגל את הנלמד בהרצאות ובתרגולים.
- נתנסה בתכנות ב-python לפתרון בעיות פרקטיות.

הנחיות כלליות

- **<u>תאריך הגשה</u>**: 7.12.2025
- את המטלה יש להגיש **<u>בזוגות בלבד</u>**.
- יש להגיש מטלות מוקלדות <u>בלבד</u> בעברית או באנגלית. פתרונות בכתב יד <u>לא ייבדקו</u>.
- שאלות בנוגע לתרגיל בכל נושא שהוא יש להעלות לפורום המיועד:
- https://piazza.com/technion.ac.il/winter2026/02360501
- המתרגלת האחראית על תרגיל: **אראל חדד**.
  - בקשות דחיה **מוצדקות** (מילואים, אשפוז וכו׳) יש לשלוח למתרגל האחראי (רועי טייכמן).
  - במהלך התרגיל ייתכן כי קבצי התרגיל (כולל מסמך זה) יעודכנו. הודעה על כך תפורסם בהתאם.
  - העדכונים הינם <u>מחייבים</u>, ועליכם להתעדכן עד מועד הגשת התרגיל.

- **שימו לב! עליכם לקרוא בעיון את מסמך מדיניות השימוש בכלי AI בתרגילי הבית, שנמצא באתר הקורס, ולענות בפירוט רב על השאלה האחרונה במסמך זה.**
- שימו לב, התרגיל מהווה כ-10% מהציון הסופי במקצוע **<u>ולכן העתקות תטופלנה בחומרה!</u>**
- ציון המטלה יורכב מהגורמים הבאים:
  - **65% - המסמך היבש**.
  - **35% - הקוד המוגש.**

- אנו יודעים שעבור חלקכם זו התנסות ראשונה בכתיבת קוד בפייתון, והתרגיל מתוכנן בהתאם לכך.
  - שימו לב כי לא יענו שאלות בסגנון "איך מוצאים את עלות הפתרון שהוחזר?" / "איך ניגשים למפה מתוך המימוש של הפונק' ההיא?" / "באיזה שדה שמור ה...?" וכדומה.
  - אנחנו רוצים לעודד אתכם לעיין בקוד ולמצוא פרטים אלו בכוחות עצמכם. הכרת סביבת העבודה שסיפקנו לכם והתמצאות בה הן למעשה חלק מהתרגיל.
  - בתרגילי הבית בקורס הרצת הניסויים עשויה לקחת זמן רב, לכן מומלץ מאוד להימנע מדחיית העבודה על התרגיל ו/או כתיבת הדו"ח לרגע האחרון. לא תינתנה דחיות על רקע זה.
- מסמך זה כתוב בלשון זכר מטעמי נוחות בלבד, אך מתייחס לכלל המגדרים.

כפי שצוין, אנחנו קשובים לפניות שלכם במהלך הזמן המוקצב להגשת התרגיל ומעדכנים את המסמך בהתאם. בעת הצורך, גרסאות עדכניות של המסמך יועלו לאתר הקורס, ולכל עדכון יצורף מספר גרסה כדי שתוכלו לעקוב. ייתכן שתפורסמנה מספר גרסאות – אל תיבהלו מכך, השינויים בכל גרסה יכולים להיות מינוריים.
שימו לב כי הבהרות ועדכונים שנוספים למסמך אחרי הפרסום הראשוני יסומנו כאן בצהוב. הנחיות **<u>מחייבות</u>** כלליות נוספות יופיעו תחת פוסט נעוץ בפורום הפיאצה בהתאם לצורך.

הנחיות לחלק היבש

1. ככלל אצבע, בהינתן שאלה ראשית ספקו את התשובה המיידית ולאחר מכן הרחיבו והסבירו. למשל, אם השאלה היא מה סיבוכיות הזמן של אלגוריתם BFS, תשובה מספקת לדוגמה תהיה "$O(b^d)$, מכיוון שבמקרה הגרוע ביותר נאחסן את כל עץ החיפוש של הבעיה ב-CLOSE".
2. ישנן שאלות עם יותר מפתרון יחיד. כל פתרון נכון יקבל את מלוא הנקודות, בהנחה שהוא אכן מדויק ומנומק היטב.

הנחיות לחלק הכולל קוד

1. אנו מעודדים אתכם לעבור על הקבצים המצורפים ולהבין כיצד הסביבה בנויה ובאילו פונקציות תוכלו להשתמש במימוש שלכם.
2. הקוד שלכם ייבדק בקפדנות על ידי טסטים. הטסטים יבדקו את הפתרונות המוחזרים על ידי האלגוריתמים שלכם אל מול המימוש שלנו על פני בעיות שונות. אנו מצפים ממכם (אלא אם צוין אחרת) להחזיר את אותם ערכים בדיוק. אנחנו נבדוק את **המסלול המוחזר**, **מספר הצמתים שפותחו** ואת **עלות הפתרון המוחזר**. שימו לב כי הטסטים יהיו מוגבלים בזמן אך תקבלו זמן גדול מאוד לכל טסט.
3. ספקו קוד ברור ונקי הכולל תיעוד המאפשר גם בדיקה ידנית.

הוראות הגשה

- עליכם להגיש קובץ יחיד בשם AI1_<ID1>_<ID2>.zip (כאשר <IDX> מתאר את ת״ז של כל מגיש/ה), המכיל:
  - קובץ בשם dry.pdf המכיל את התשובות לחלק היבש.
  - קובץ בשם Algorithms.py המכיל את המימוש שלכם לאלגוריתמי החיפוש.

מבוא ורקע

תרגיל הבית מתפרש על פני מסמך זה והמחברת המצורפת – **236501_IAI_Win26_HW1.ipynb**. מומלץ בחום לענות על השאלות לפי הסדר המתואר במסמך זה.

במטלה זו נעסוק בהפעלת אלגוריתמי חיפוש על מרחבי מצבים לבעיות ניווט. מומלץ לחזור על שקפי ההרצאות והתרגולים הרלוונטיים לפני תחילת העבודה על התרגיל. במהלך התרגיל תתבקשו להריץ מספר ניסויים ולדווח על תוצאותיהם. לשם כך תידרשו לבצע ניתוח של התוצאות שהתקבלו, כפי שיוסבר בהמשך המסמך.

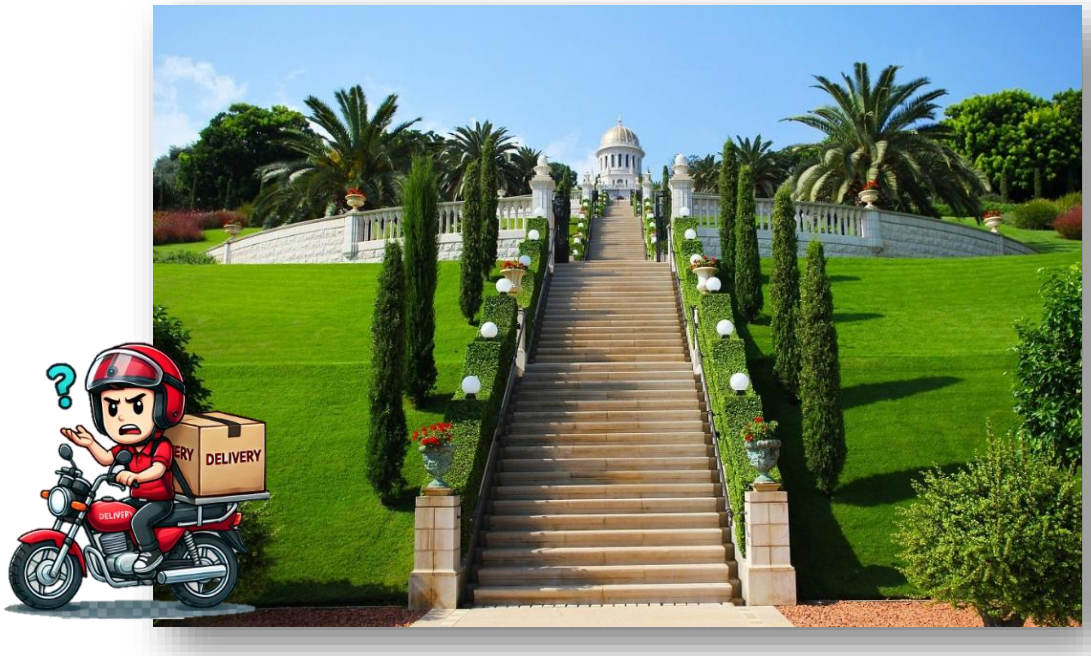
סיפור מסגרת

תומר וספיר, זוג חברים חיפאיים שלכם, הם בעלים של מסעדה שמציעה אוכל ג׳מייקני מיוחדת מסוגה בארץ.

לאחר שחוו ירידה משמעותית בהכנסות, החליטו תומר וספיר לצאת לדרך יזמית חדשה – ולהקים חברת שליחויות אוכל משלהם, ***BOLT***, שתתחרה בשאר החברות בעיר.

הזוג כבר הלך לרשום את BOLT ברשם החברות, שכר שליחות ושליחים, ומזה כמה שבועות מפעילים את השירות בעיר ובסביבה! **אז מה הבעיה, אתם שואלים?** שיבושי ה-GPS באיזור חיפה והמפרץ גרמו לכך שיהלי, השליח החדש שלהם, אף פעם לא מגיע ליעד בזמן...

תומר וספיר, שיודעים שאתם סטודנטים מצטיינים למדעי המחשב בטכניון, ביקשו מכם לדבר עם יהלי ולעזור לו למצוא את הדרך להגיע מהמסעדה אל כתובת היעד. כשיהלי מתקשר אליכם (בוכה וממלמל שהוא לא יכול להרשות לעצמו שיפטרו אותו), אתם מרגיעים אותו שלא רק שאתם יודעים את הדרך, אלא שאחרי שכבר למדתם אלגוריתמים רבים בקורס מבוא לבינה מלאכותית – **גם תעזרו לו לתכנן את המסלול הטוב ביותר להגיע מהמסעדה ליעד.**



הגדרת סביבת העבודה עם Jupyter

לפני שתתחילו לפתור את התרגיל, נבקש **שתקדישו מספר דקות** לוודא כי סביבת העבודה שלכם מוגדרת היטב, לפי ההוראות המובאות כאן.
לאורך התרגיל נעבור לסירוגין בין עבודה על שאלות תיאורטיות לבין מימוש האלגוריתמים שלמדנו בכיתה עבור פתרון הבעיה אשר הוצגה במבוא.
על מנת לאפשר עבודה נוחה וממוקדת במימוש האלגוריתמים עצמם, אנו מספקים לכם את המחברת **236501_IAI_Win26_HW1.ipynb** המאפשרת הרצה נוחה של האלגוריתמים השונים וכן ויזואליזציה של פעולתם.
המחברת הנ"ל היא מסוג Jupyter Notebook, המאפשרת הרצת קוד פייתון באופן שמותאם לניסויים כך שניתן לשנות שינויים מינוריים בקוד וכן להריץ מספר רב של ניסויים בזה אחר זה באופן אינטראקטיבי. הקובץ הנ"ל המשמש כמעין "מחברת" שיאפשר לכם להריץ את הניסויים שלכם לאורך התרגיל בצורה קלה.
על מנת להימנע מתקלות הנובעות מהגדרת סביבת הפייתון שבה תעבדו, אנו ממליצים לעבוד עם **Google Colab**. כדי לאפשר עבודה בפלטפורמה עליכם להעלות את תוכן התיקייה של התרגיל לתוך תיקייה ב-Drive האישי שלכם, ולאחר מכן לפתוח את המחברת (קובץ ה-.ipynb) ולפעול על פי ההוראות. צפו בבקשה **בסרטון ההדרכה הבא** על מנת למנוע אי-נעימויות בכל הקשור לתקלות הנובעות מהגדרת הסביבה.
שימו לב כי גרסת הפייתון איתה אתם נדרשים לעבוד היא **python 3.10**. גרסאות אחרות עלולות להוביל לתקלות שנובעות מאי-תאימויות במהלך ההתקנה של הספריות הנוספות הנדרשות בתרגיל שאינן נכללות בגרסת הפייתון הבסיסית, בפרט:

Gym
Numpy
Heapdict
Typing

- **אין לעשות** import לספריות שלא הוגדרו בקוד שסופק לכם ושלא ניתן להן אישור מפורש בפורום **הפיאצה**.
- התחילו לענות על חלק זה רק לאחר שהגדרתם את סביבת העבודה והבנתם כיצד להריץ את הקוד במלואו.
- מומלץ לעבור על הקוד במחברת במקביל למסמך היבש, ולענות על השאלות השונות שלב-שלב.

מרחב החיפוש

השאלות בחלק זה מתבססות על המפה "8x8" שמופיעה במחברת, אלא אם נכתב אחרת.

שאלה 1 – מבוא והיכרות עם מרחב החיפוש (9 נק'):

S	F	F	F	F	F	F	F
F	F	F	F	F	T	A	L
T	F	F	**H**	F	F	T	F
F	**P**	F	F	F	**H**	T	F
F	A	F	**H**	F	**P**	F	F
F	**H**	**H**	F	F	F	**H**	F
F	**H**	T	F	F	F	T	L
F	L	F	**H**	F	F	F	**G**

1. **קוד**: עברו על המחברת עד שאתם מגיעים לחלק של BFS-G ועצרו שם (לפני שתממשו את האלגוריתם הראשון במחברת).
השאלות בחלק זה מתבססות על המפה "8x8" שמופיעה במחברת (וכן במסמך הנ"ל), אלא אם נכתב אחרת.
2. תחילה נרצה להגדיר את מרחב החיפוש כפי שנלמד בתרגול. נגדיר:

$S = X^d$ = מרחב המצבים, אשר כל מצב בו מתואר על ידי וקטור של d פיצ'רים

O = מרחב האופרטורים

I = המצב ההתחלתי

G = קבוצת מצבי המטרה

הגדירו את (S, O, I, G) עבור סביבת חיפה. (1 נק')

State is defined as the state index on the map that is indexed from 0 to 63
Each state corresponds to one cell in the 8×8 map, indexed as:

$$\text{state} = row \times 8 + col$$

S = { 0,1, .., 63}
O = {0,1,2,3} = { up ,down, left, right}
I = 0
G = 63

3. מה גודל מרחב המצבים S? הסבירו. (1 נק')

$$|S| = 64$$

4. מה הוא מקדם הסיעוף בבעיית הניווט בחיפה? (1 נק')

In Haifa, the agent can try **4 actions** from any square so b = 4

5. האם קיימים מעגלים במרחב החיפוש שלנו? (1 נק')

Yes moving back and forth between squares or using passways can create cycles, An agent can go lest then right then left and he will return to the same place he was firstly in such that he will create a circle.

6. מה תחזיר לנו הפונקציה Succ על המצב 3 (כפי שמוגדר במחברת)? (1 נק')

It returns a dictionary of all possible actions from that state, since state 3 is at the first raw and the 4th col the agent cant go up , so this function will return left, right, down

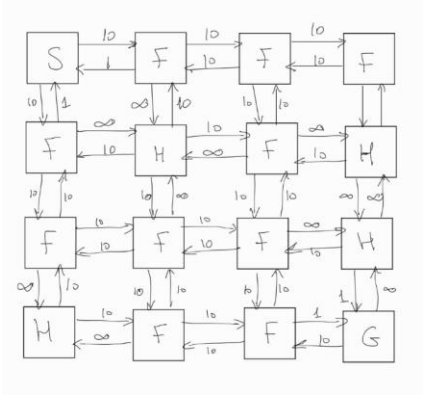
7. מה תחזיר לנו הפונקציה Domain על האופרטור DOWN (אופרטור 0)? (1 נק').

$$domain(down) = \{s \in S \mid raw(s) < 7\}$$

8. במקרה הגרוע ביותר, כמה פעולות ידרשו לסוכן Random Agent להגיע למצב הסופי? ובמקרה הטוב ביותר? (1 נק')

At the worst case Random agent can get stuck in a circle movement so he will need infinite action and will never get to the goal, but at the best case scenario he can choose the Manhattan path so in this case where the goal is at the 63 state he will need to go to the 8th raw and the 8th col so 15 actions needed in total.

9. עבור **המפה "4x4" שמופיעה במחברת**, ציירו את גרף המצבים. חשבו על כלל הרכיבים שיש להתייחס אליהם בגרף שכזה. (1 נק')



😊 and we need to add that each state in indexed accordingly

10. עבור מפה כללית בסביבת חיפה, בה יכולים להיות מספר מצבי מטרה (לדוגמה במקרה שבו יהלי יתרצה גם אם יגיע הביתה ויאכל את האוכל מהמשלוח בעצמו), האם המסלול הזול ביותר (מבחינת עלות המסלול) הוא גם המסלול שמגיע למצב המטרה הקרוב ביותר למצב ההתחלתי (במונחים של Manhattan Distance, כפי שנלמד בכיתה)? אם כן, הוכיחו. אם לא, ספקו דוגמה נגדית. (1 נק')

No because the cheapest path is not necessary closest goal in Manhattan distance,

Map:

S P P G1

F F F G2

F F F F

In Manhattan distance the goal G1 is closer that the goal G2 $3 < 4$, but G1 cost 101 and G2 costs 31

חיפוש לא מיודע

השאלות בחלק זה מתבססות על המפה "8x8 שמופיעה במחברת, אלא אם נכתב אחרת.

שימו לב – כאשר אתם נשאלים לגבי היווצרות ופיתוח של צמתים בעת הריצה של אלגוריתם כלשהו:

- הניחו כי צומת התחלתי נוצר אוטומטית עם תחילת הריצה של האלגוריתם.
- התייחסו לסדר היצירה והפיתוח של הצמתים תוך כדי ריצת האלגוריתם כפי שהוא מוגדר לפי הפעולות השונות במחברת-
  - Down – 0 (יפותח ראשון)
  - 1 - Right
  - 2 - Up
  - Left – 3 (יפותח אחרון)

שאלה 2 – Breadth First Search (BFS) (15 נק'):

1. קוד: ממשו את הפונקציות הנדרשות עבור ריצתו של האלג' BFS-G בקובץ Algorithms.py, ע"פ ההנחיות המופיעות במחברת. (10 נק')
2. מה צריך להיות התנאי על גרף החיפוש (לא בהכרח בבעיית הניווט בחיפה) כך שאלג' BFS (שרץ על עץ) ואלג' BFS-G (שרץ על גרף) ייצרו ויפתחו צמתים זהים באותו הסדר? (1 נק')

The condition is for BFS on a tree and BFS-G on a graph to expand nodes in the same order, no two nodes at the same depth in the search tree may share the same successor.
In BFS on a tree, each node is expanded independently, so the same successor could appear multiple times.
In BFS-G, a visited state is only expanded once , so if two "brother" nodes share a successor, BFS-G will skip the second expansion.

3. נתונה מפה בגודל NxN שלא מכילה Passways. הציעו דרך להשתמש באלגוריתם BFS-G כך שיחזיר פתרון אופטימלי (עלות מינימלית) והסבירו בקצרה. (2 נק')

- **רמז:** מצאו פונקציה $T: G \rightarrow G'$ המקבלת את גרף המצבים G ויוצרת גרף חדש G', ובעזרתה למצוא את המסלול האופטימלי בגרף G.

The idea is since Bfs gets the shortest path , and we have a weighted path , if an edge costs n>1 , we can replace it with n vertices that are connected with edges weighted by 1 (they are not connected to any other vertices).and then we can runc Bfs

To use BFS-G to find a **minimum-cost path**, we can transform the original weighted graph G into a new unweighted graph G' as follows:

- For each edge in G with cost $n > 1$, replace it with a **chain of n new intermediate vertices** connected by edges of cost 1.
- These intermediate vertices are only connected along this chain, not to any other vertices.
- Edges of cost 1 remain unchanged.

Now, BFS-G on G' will find the **shortest path in terms of edges**, which corresponds exactly to the **minimum-cost path in** G.

4. נתונה מפה בגודל NxN, ללא חורים, ללא Passways, המכיל $N^2 - 2$ משבצות רגילות (F,T,A,L), מצב התחלתי בפינה השמאלית עליונה ומצב מטרה בפינה הימנית תחתונה. כמה צמתים יפותחו וייווצרו במהלך חיפוש BFS-G? הסבירו והסתמכו בתשובתכם על פי הפסאודו-קוד וההגדרות שראינו בתרגולים. (2 נק')

BFS-G avoid expanding(calling for succ func) for the same node , the expansion starts from the S node and the only way to get to the goal is by going left or right therefore each node at a diagonal form will get expanded , but when we reach the last diagonal before the goal node , we will only need to expand one of these two nodes , cause the goal node will be explored and returned as the output once we expand on of these 2 states , this means that only 2 nodes won't be expanded on the map(one of the last diagonal nodes and the goal node) -> N^2-2 nodes will be expanded

In the map all of the states will be explored (called for make node func) and since we are in BFS_G we won't call foe make node func on the same state more than once cause it will be in the closed set , but we will explore all the states up to the goal state ->N^2 will be explored.

שאלה 3 – Depth First Search (DFS), וריאציות על DFS (10 נק'):

1. עבור בעיית הניווט בחיפה עם מפה NxN, האם האלגוריתם DFS-G שלם? האם הוא קביל? (1 נק')

On an NxN map DFS_G is complete because it will not get stuck in a cycle , and since N is an infinite number the algorithm will finish running

DFS_G is not admissible it might reach the goal through a not optimal path.

2. עבור בעיית הניווט בחיפה, נתונה מפה בגודל NxN. האם אלגוריתם DFS (שרץ על עץ), עבור בעיית הניווט בחיפה על מפה NxN, היה מוצא פתרון כלשהו? אם כן, מה המסלול שיתקבל? אם לא, כיצד האלגוריתם היה פועל? (1 נק')

DFS-T is not complete because it might get stuck in a cycle , if we go right->down->left->up we will get to the same state we started in and stuck forever.

3. כעת נתונה מפה בגודל NxN, ללא Holes וללא Passways. המפה מכילה $N^2 - 2$ משבצות "רגילות" (F/T/A/L), מצב התחלתי S בפינה השמאלית העליונה שלה ומצב מטרה G בפינה הימנית התחתונה שלה.

1. כמה צמתים יפותחו וייווצרו במהלך חיפוש DFS-G? (1 נק')

The algorithm will expand all the nodes up the left corner of the last line by a Down movement ($N^2 - N$) , and then will go Right all the way to the goal but won't expand the goal because it will be explored in the state N^2-1, so the total of expanded states is 2N – 2 .

The algorithm will explore (call for make node) on 4N – 5 without exploring a same node more than once , each node that is expanded it will explore the nodes linked to it

At first DFS goes from the start node to the $N^2 - N$ node so it will explore 2N -1 (Right and Down)

The N^2 - N node will only explore the node on the right

The N^2 - N -1 node will only explore the node on the right (cause the up one was already explored)

And then from $N^2 - N$ -2 node to $N^2 - 1$ each node will explore 2 nodes (Right and up) -> 2N -4

In total 4N -5

2. כמה צמתים יפותחו וייווצרו במהלך חיפוש DFS-G **backtracking**? הסבירו בתשובתכם מהו היתרון בשימוש באלג' זה על פני שימוש באלג' DFS-G מתת הסעיף הקודם. (1 נק')

Backtracking DFS_G will expand the same states 2N – 2

But it only explores(make node func) the nodes that it will use it in the solution path therefore 2N – 1 (including the goal node).

4. יהלי רוצה למצוא מסלול בסביבת חיפה עם DFS-L. ידוע כי אורך המסלול הקצר ביותר לצומת מטרה הוא d אך יהלי מחליט להגביל את עצמו לחיפוש בעומק $\frac{d}{2}$ בלבד.

1. עבור מפה כללית בגודל NxN, הציעו שינוי לבעיית החיפוש **ברמת מרחב החיפוש** (S, O, I, G) כך שיהלי יוכל למצוא פתרון מבלי להפר את מגבלת העומק. הסבירו למה כעת ניתן למצוא פתרון. (3 נק')

The idea is that at each step we do 2 movements so we need to change the Operator,

$$O' = \{ (o1, o2) |\ oi \in \{up, down, left, right\}\}$$

The shortest real path has length d, but in the new compressed search space its length becomes d/2, which is at or below Yahli's depth limit

Therefore DFS-L can now reach the goal without increasing the depth bound

2. האם השתנה מקדם הסיעוף? אם כן, מה מקדם הסיעוף החדש b'? רשמו את התשובה כתלות בb (מקדם הסיעוף בבעיה המקורית). (1 נק')

The new branching factor is $b^2 = b'$

3. מהן סיבוכיות הזמן והמקום החדשים? ענו במונחים של b, d והשוו את התשובה ל-DFS-L רגיל עם עומק d. (1 נק')

Time complexity = $o(b^{2})^{(d/2)}$

Space complexity =$o(b^{2} * d/2) = 0(b^{2} * d)$

4. ספקו דוגמה לבעיה שבה DFS-L במרחב החיפוש החדש (לאחר השינויים שביצעתם בסעיף 4.1) טוב יותר מאשר DFS-L במרחב החיפוש הקודם, ודוגמה לבעיה שבה DFS-L במרחב המקורי טוב יותר (במונחים של מספר הצמתים שפותחו, והתייחסו למספר זה בתשובתכם). (מרחב החיפוש הקודם הינו עם עומק d ומרחב החיפוש החדש הינו עם עומק $\frac{d}{2}$)

דוגמות יכולות להיות כלליות ולא בהכרח מסביבת חיפה. (1 נק')

The DFS_L with depth d/2d is better when the goal could be found mostly at a shallower depth for example goal is at a depth k < d/2 but it's a the right most of the DFS tree so it will be explored last , original dfs-l will search all the tree up to depth d , but the updated will search roughly a tree with depth d/2 and find the solution.

The DFS_L with depth d is better if the goal could be found mostly at a depth k where d/2 < k < d and at the leftmost of the DFS tree , so DFS_L with depth d/2 can't find the goal but with depth d can find it and quickly.

שאלה 4 – Uniform Cost Search (UCS) (4 נק'):

1. האם UCS שלם וקביל אם קיימת קשת אחת שעלותה שלילית אך לא קיימים מעגלים שליליים? נמקו והסבירו מה עלול להשתבש (1 נק')

At this case UCS would still be complete because it will not get stuck at a negative circle , but it will not be admissive cause its optimality comes from the optimality of Dijkstra in case the cost function is bounded with k > 0.

2. עבור אילו בעיות חיפוש אלגוריתם UCS ואלגוריתם BFS-G יפעלו באותו האופן, כלומר יחזירו אותם המסלולים כפתרון? הסבירו. (1 נק')

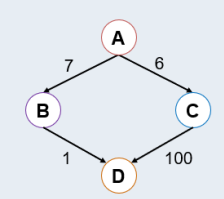
When the cost function is constant equal to j on all the edges

3. יהלי טעה במימוש של אלגוריתם UCS ובדקה בטעות בזמן יצירת הצומת האם הוא צומת מטרה במקום בזמן הפיתוח שלו (כלומר, לאחר הוצאתו מתור העדיפויות). תנו דוגמה לגרף חיפוש שעבורו האלג' שיהלי מימש יחזיר בכל זאת את המסלול הקל ביותר, ודוגמה לגרף חיפוש שעבורו האלג' לא יחזיר את המסלול הקל ביותר. (2 נק')
   - עבור כל דוגמה, הסבירו מה המסלול והעלות ש-UCS השגוי החזיר, ומה המסלול והעלות שהאלגוריתם הנכון היה מחזיר. נדגיש שגרף החיפוש לא בהכרח צריך לייצג את בעיית הניווט בחיפה – ניתן לתת דוגמה לגרף שמייצג בעיית חיפוש אחרת.
   - על הגרפים המוצעים להכיל קשתות מכוונות וכן את העלות של כל קשת.

A graph that will not return the optimal answer:

An execution of the wrong UCS will output the path A->C->D with a cost of 106 which is not optimal

But the UCS algo will output the optimal path A->B->D



But in case the cost of the edge (c,d) = 0.5 both of the algorithms will output the same answer A->C->d

חיפוש מיודע

השאלות בחלק זה מתבססות על המפה "8x8" שמופיעה במחברת, אלא אם נכתב אחרת.

שאלה 5 – יוריסטיקות (8 נק'):

1. תהיינה שתי יוריסטיקות עקביות h_1, h_2. שימו לב: אם בחרתם להפריך, תוכלו לתת דוגמא נגדית שלא קשורה למפה שבמחברת.
   1. הוכיחו/הפריכו: בהינתן $\alpha, \beta \geq 1$, היוריסטיקה $h = \alpha h_1 + \beta h_2$ עקבית. (1 נק')

Counter example :

graph with two nodes S and G and one edge $S \to G$ with the cost $c = 1$ and 2 heuristics h1 , h2 that are consistent :

$$h_1(G) = h_2(G) = 0, h_1(S) = h_2(S) = 1$$

Each $h_i$is consistent: $h_i(S) = 1 \leq 1 + 0 = h_i(G) + c$.

Define $\alpha = \beta = 1$ Then we get

$$h(S) = \alpha h_1(S) + \beta h_2(S) = 2 \quad h(G) = 0$$

for h to be consistent we need $h(S) \leq c + h(G) = 1 + 0 = 1$

but $2 \leq 1$ so h is not consistent

2. הוכיחו/הפריכו : בהינתן $0 < \alpha < 1$, היוריסטיקה $h = \alpha h_1 + (1 - \alpha)h_2$ עקבית. (1 נק')

Prove:

$h_1$and $h_2$are consistent :

$$h_1(n) \leq c + h_1(n')$$
$$h_2(n) \leq c + h_2(n').$$

Multiply the first function by α and the second by $(1 - \alpha)$ and add them together :

$$\alpha h_1(n) + (1 - \alpha)h_2(n) \leq c\alpha + \alpha h_1(n') + c(1 - \alpha) + (1 - \alpha)h_2(n').$$

$$c\alpha + c(1 - \alpha) = c$$
$$h(n) \leq c + h(n').$$

Then by definition h is consistent.

2. תהיינה שתי יוריסטיקות h_1, h_2.

בהינתן ש h_1 קבילה אך h_2 אינה קבילה, האם יש דרך לשלב בינהן כך שהתוצאה תהיה קבילה? נמקו את תשובתכם (2 נק')

Yes there is away by defining a new heuristic $h(n) = \min\{h_1(n), h_2(n)\}$

Since $0 \leq h_1(n) \leq h^*(n)$

And h_2 might have some nodes such that $h_2(n) > h^*(n)$

Then by definition h is consistent.

3. הציעו יוריסטיקה נוספת, המתבססת על h_1 ו-h_2 קבילות ועקביות כלשהן, בה נוכל להשתמש בבעיית חיפוש כללית כלשהי. הסבירו בקצרה מדוע אולי נרצה להשתמש בה, ואיזו תכונה בה באה לידי ביטוי. (2 נק')

Define the new heuristic $h(n) = \max\{h_1(n), h_2(n)\}$

This heuristic is consistent and admissible and it can give us the information that is dominant in both of $h_1 and\ h_2$ since we are getting the max from the both worlds for example if both of the heuristics are about distance and since they are admissible if we get the max of them we will get the closer value to $h^{perfect}$.

4. נגדיר יוריסטיקה חדשה עבור בעיית הניווט בחיפה:

$$h_{HAIFA}(s) = \min\{\, min\{h_{Manhatan}(s,g) \mid g \in G\,\}, C_{passway}\}$$

כאשר הביטוי $h_{Manhatan}(s,g)$ מתאר את מרחק מנהטן מהמצב הנוכחי למצב סופי, והביטוי $C_{passway}$ מתאר את מחיר השימוש ב-Passway (לדוגמה, 100 בבעיית הניווט בחיפה). שימו לב כי היוריסטיקה מחשבת את מרחקי מנהטן מהמצב הנוכחי למצב סופי על פני כל צמתי היעד.

האם היוריסטיקה h_{HAIFA} עקבית עבור כל מפה בבעיית הניווט בחיפה? אם כן, הסבירו בקצרה. אם לא, ספקו דוגמה נגדית. (2 נק')

Yes h_haifa is consistent for all maps ,

The idea is that h_manhatten is not consistent on Haifa env because there exists the passway that teleport us from a place to another on the map that may increase the manhatten distance by a lot but the cost of this transport is const = 100

, $h_{manhatten}(s) > cost_{passaway} + h_{manhatten}(s')$

But at this case the h_haifa would pick the C_passway as its value because the h_manhatten would diffinitly be bigger than the cost of passway,

Now lets divide into cases:

H_haifa = min{h_manhatten(s,g) } = h_min implement

We can conclude that we didn't go through a passway cause h_min is smaller than the c_passway thus h_min is consitant because the cost from s to s' can't be smaller than the manhatten distance between them which is 1 .

H_haifa = C_paasway

We can conclude that we went through a passway and the h_manhatten was not consistent at this move (the case we described before), but at this move h_haifa is consistent because

$$h_{haifa}(s) = C_{passaway} = 100 \leq cost\ (s,s') + h_{haifa}(s') = 100 + \ h_{haifa}(s')$$

Therefore h_{haifa} is consistent.

שאלה 6 – Greedy Best First Search (13 נק'):

1. קוד: ממשו את הפונקציות הנדרשות עבור ריצתו של האלג' Greedy Best First Search בקובץ Algorithms.py, ע"פ ההנחיות המופיעות במחברת. השתמשו ביוריסטיקה h_{HAIFA}. (10 נק')

2. האם האלג' Greedy Best First Search, על מפה כללית עבור בעיית הניווט בחיפה בגודל NxN, הוא שלם? האם הוא קביל? (1 נק')

The Haifa map env is finite state space NxN and it is connected (the holes do not disconnect reachable states from start to goal) so the algorithm is complete

But it still ignores the path cost so it is not optimal

3. יהלי טוען שאלג' Beam Search כאשר $K = 1$ למעשה שקול לאלג' Greedy Best First Search. האם יהלי צודק? הסבירו בקצרה. (2 נק')

Yes because when k = 1 Beam Search keeps only the single best node at each level which is exactly what Greedy Best-First Search does.

Therefore, Beam Search with k = 1 is equivalent to GBFS

שאלה 7 – A*, וריאציות על A* (21 נק'):

1. **קוד**: ממשו את הפונקציות הנדרשות עבור ריצתו של האלג' $A^* - \epsilon$ בקובץ Algorithms.py, ע"פ ההנחיות המופיעות במחברת. גם עבור אלגוריתם זה, השתמשו ביוריסטיקה h_{HAIFA} (כפי שכתוב גם במחברת, השתמשו באותה ההיוריסטיקה כשובר שוויון בכלל ההחלטה כשנדרש) (10 נק')

2. בכיתה הגדרנו את פונקציית ההערכה עבור הצומת הבא לפיתוח במהלך ריצת האלג' A^* באופן הבא:

$$f(v) = h(v) + g(v)$$

יהלי טוען כי אפשר להשתמש בפונקציה $f'(v) = \frac{h(v)+g(v)}{2}$ במקום ב-$f(v)$, ותוצאת הריצה של האלג' תהיה שקולה.

הסבירו בקצרה מדוע יהלי צודק (בתשובתכם התייחסו לסדר פיתוח הצמתים, המסלול המוחזר ועלות המסלול המוחזר מהאלג' בעת השימוש בפונקציה $f'(v)$). (2 נק')

According to Astar epsilon , first we get the f_{min} from the open list that it doesn't change with multiplying with 0.5 all the f values of the open list

And then we build the focal list which is nodes from the open within arrange of $f(n)/2 \leq (1+\varepsilon)\cdot fmin/2$ so it ends up to be the same equivalent that we used in the original algorithm and we end up choosing the same nodes to expand , that be said the optimal cost and the path chosen won't be changed .

3. **קוד**: ממשו את הפונקציות הנדרשות עבור ריצתו של האלג' A^* בקובץ Algorithms.py, ע"פ ההנחיות המופיעות במחברת. גם עבור אלג' זה, השתמשו ביוריסטיקה h_{HAIFA}. (5 נק')

   - **רמז:** חשבו כיצד תוכלו לממש את האלג' A^* ע"י שימוש במימוש שהצעתם עבור $A^* - \epsilon$.

4. נסתכל על היוריסטיקה קבילה h. הוכיחו כי אלגוריתם $A^* - \epsilon$ מחזיר פתרון שעלותו C_ϵ כאשר: $C^*(\epsilon + 1) \geq C_\epsilon \geq C^*$ כאשר C^* הינו המחיר של המסלול האופטימלי . השתמשו בכך ש h קבילה (מהי ההגדרה להיורסטיקה קבילה?) , הסתכלו על צומת המטרה שתימצא. (3 נק')

C^*

is the minimum possible cost to any goal, so any returned path cost $C_\varepsilon$cannot be smaller than C^*Thus $C_\varepsilon \geq C^*$

Using Astarepsilon algorithm it gives us an upper bound of the cost that it return which is $C^*(\epsilon + 1)$

prove:

When the algorithm expands the goal t we have t in the focal set, so

$$C_\varepsilon = g(t) = f(t) \leq (1 + \varepsilon) f_{\min}.$$

the heuristic is admissible let n be an optimal path to the goal

$$f(n) = g(n) + h(n) \leq g(n) + h*(n) = cost\ of\ that\ path \leq C*.$$

Therefore $f_{\min} \leq f(n) \leq C^*$.

Combine $C_\varepsilon \leq (1 + \varepsilon) f_{\min} \leq (1 + \varepsilon) C^*$.

And we get

$$C^* \leq C_\varepsilon \leq (1 + \varepsilon) C^*.$$

5. כיצד נוכל להפוך את אלגוריתם $A^* - \epsilon$ לאלגוריתם anytime? חשבו על הדיון שערכנו בכיתה לגבי המשמעות של סוגי האלגוריתמים הללו (1 נק')

I think that if each iteration of the while (open) loop we gradually reduce the value of epsilon we can restrict the algorithm each time a little bit more , so when we get more time to run the algorithm we get closer to the optimal path.

At first we run with a large ε so the algorithm quickly finds a (possibly non-optimal) solution.

Then, as time allows, we decrease ε and continue

Thus, as time progresses, the algorithm keeps improving the solution until it eventually reaches the optimal path.

ניתוח התוצאות

שאלה 8 – Benchmarking (5 נק׳):

בשאלה זאת נשווה בין אלגוריתמי החיפוש השונים שמימשתם על בעיות שונות.

1. **קוד**: הריצו את החלק הרלוונטי במחברת. וודאו כי אתם רואים את קובץ ה-csv שנוצר (תוכלו לפתוח אותו עם Excel או כל תוכנה אחרת מתאימה).
2. **הסבירו את התוצאות.** האם הן תואמות לציפיות שלכם? האם התוצאות היו משתנות עם יוריסטיקה מיודעת יותר? נתחו והסבירו את התוצאות במונחים של מספר פיתוחים, המסלול המוחזר ועלות הפתרון (מומלץ לצרף גם ניתוח ויזואלי כלשהו, כגון גרף או טבלה).
   שימו לב שבסעיף זה אין תשובה נכונה או לא נכונה, אך אתם נדרשים לספק הסבר מפורט ומבוסס. (5 נק׳)

E	D	C	B	A
Greedy Search num of expanded nodes	Greedy Search cost	BFS-G num of expanded nodes	BFS-G cost	map
33	227	141	109	map12x12
54	188	223	121	map15x15
46	268	390	256	map20x20
A*-Epsilon (Epsilon=0.5) num of expanc	A*-Epsilon (0.5) cost	A* num of expanded nodes	A* cost	
95	87	98	87	map12x12
161	115	168	106	map15x15
455	175	309	175	map20x20
A*-Epsilon (Epsilon=4) num of expande	A*-Epsilon (4) cost	A*-Epsilon (Epsilon=2) num of expanded	A*-Epsilon (2) cost	
27	89	27	89	map12x12
42	138	77	106	map15x15
42	206	2264	175	map20x20

Cost :
BFS_G gives us cost 109 -> 256 which is much higher than the informed algorithms because they use heuristics to explore better paths

Greedy search cost only follows the heuristic and ignores the cost , so it has higher costs among all the algorithm

A star finds optimal paths with lower costs

A star epsilon finds good enough cost paths better than BFS but its not optimal like A star , and with epsilon increasing (cause we want to find a path faster) the cost is increasing too but its not that bad cause its bounded with epsilon.

Expanded nodes:
Bsf expands a lot of nodes comparing to greedy because bfs explores all nodes uniformly

A star expands more nodes than Greedy because it considers both path cost and heuristic
A star epsilon :
Small epsilon (0.5) → behaves close to A* (similar expanded nodes and cost) cause it chooses the nodes from the Focal list that most likely A star chooses
Larger epsilon (2 or 4) → fewer expansions but higher costs, especially for larger maps.

חיפוש לוקאלי

שאלה 9 – Local Search (10 נק׳):

עכשיו כשאתם ויהלי כבר מכירים קצת, אתם מגלים שהוא בחור קצת מבולגן.
לא רק שהוא טועה בכבישים במהלך השליחויות, גם הכתובות למשלוח שהוא מקבל לטלפון שלו מתקלקלות לו! אז עכשיו, כשהוא כבר חבר שלכם, יהלי מבקש את עזרתכם (בתור מומחי אלגוריתמי בינה מלאכותית) לסדר את ההזמנות.
כשאתם מסתכלים בטלפון של יהלי על המסמך המכיל את כל הכתובות, אתם רואים שבאמת כל המילים בו התבלבלו... למשל, אם הופיע במסמך המקורי המשפט: ״תעשה משלוח למעונות קנדה״, במסמך שעל הטלפון של יהלי יופיעו המילים בסדר אחר, למשל: ״למעונות משלוח קנדה תעשה״.

הערות:

- הניחו כי במסמך המקורי כל מילה מופיעה פעם אחת בלבד.
- המילים מתערבבות בכל המסמך ולא רק בתוך כל משפט. הפרמוטציות של המילים במסמך הינן רנדומליות לחלוטין (ולא קורות לפי איזה סדר קבוע כלשהו).

מכיוון שאתם לא יודעים את איך נראו הכתובות במסמך המקורי, אתם לא מצליחים לסדר את המילים בסדר הנכון לבדכם. למזלכם, תומר וספיר (שיודעים את הסידור המקורי) נותנים לכם פונקציית ערך (utility function) המקבלת את המסמך ומחזירה את מספר המילים שנמצאות במיקומן המקורי במסמך (ערכים בין 0 ל-n), וכן אופרטור הלוקח שתי מילים (לאו דווקא רצופות) ומחליף בין מיקומיהן.
אחרי כל מה שלמדתם בהרצאות ובתרגולים של ״מבוא לבינה מלאכותית״, אתם רוצים להפעיל אלגוריתם חיפוש לוקאלי אשר משתמש בפונקציית הערך ובאופרטורים הנ״ל שהפתרון שלו יהיה הסידור הנכון של המילים במסמך.

1. כיצד יש להגדיר את המצבים במרחב החיפוש? (2 נק׳)

   Each state represents an ordering for the n words in the file

2. מהו מספר המצבים במרחב החיפוש? (1 נק׳)

   For n words in a file there is $n!$ permutation so if each state is a specific permutation for the n words then there is $n!$ States in total.

3. אתם נזכרים כי ספיר היא בוגרת הקורס (ואף קיבלה ציון 100 בתרגיל בית 1), לכן מבקשים את עזרתה גם בהפעלת האלגוריתם. היא מציעה לכם להשתמש ב-SAHC (Steepest Ascent Hill Climbing), על מנת למצוא פתרון. האם האלגוריתם Steepest Ascent Hill Climbing בהכרח ימצא פתרון? (2 נק׳)

   No, because as taught in the tutorial SAHC is a local search algorithm and it can get stuck if it is in a state where no single swap can improve the utility , like some words are in the right place but any single swap does not increase the number of words in the right place then SAHC gets stuck

4. תומר מגלה אף הוא על התקלה במסמך הכתובות, ומייעץ לכם להשתמש ב-SAHC with sidesways steps.
   1. האם האלגוריתם SAHC with sidesways steps בהכרח ימצא פתרון? (2 נק׳)

      It still can get stuck in a local maximum , that is lower than the global maximum but still higher than all of its neighbors , Like a state where the ordering in the file is good but we still can do better (not every word in the right place but we still can get some addresses right), but at this state any single swap just lowers the utility (locally), then SAHC with sideway steps get stuck.

   2. תומר טוען כי עבור המקרים בהם SAHC ו-SAHC with sideways steps מוצאים פתרון, עדיף להשתמש ב-SAHC with sideways steps. האם תומר צודק? (3 נק׳)
      - אם שימוש ב-SAHC with sidesways steps ימצא פתרון בפחות צעדים או ביותר צעדים, הסבירו מדוע.
      - אחרת, הוכיחו שתחת כל סדר בחירה של פעולות משפרות שני האלגוריתמים יגיעו לפתרון האופטימלי תוך מס׳ צעדים זהה. הקפידו על הוכחה פורמלית.

Tomer is not correct , SAHC if it finds a solution then it always takes the next improving of the utility step , lets say it took K steps to find the solution,
On the other hand SAHC with sideways steps , is more eager to find a solution so it always expects the worse and explore steps than may not lead to improve the utility so it takes $\geq k\ steps$, cause it may move along neighbors of equal utility.

גילוי נאות

שאלה 10 – שימוש בכלי AI לפתרון תרגיל הבית (5 נק׳):

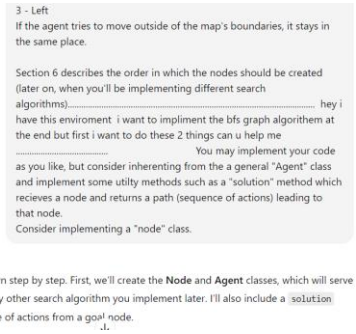
כפי שפורט במסמך המדיניות, אנו רואים חשיבות מרכזית בפיתוח **חשיבה ביקורתית** ובשימוש רפלקטיבי בכלי AI. אנו מצפים כי השימוש שתבצעו יהיה לצורך **העצמת הלמידה והבנייה המעמיקה של הידע**, ולא לצורך העתקה עיוורת או הימנעות ממאמץ קוגניטיבי.
בהמשך לכך, בשאלה זו תצטרכו לציין באילו כלי AI השתמשתם, ולפרט **בהרחבה** כיצד השתמשתם בהם, בכל אחד מחלקי התרגיל:

1. **בחלק התיאורטי ("היבש") - (2.5 נק׳)**
2. **בחלק המעשי ("הרטוב") - (2.5 נק׳)**

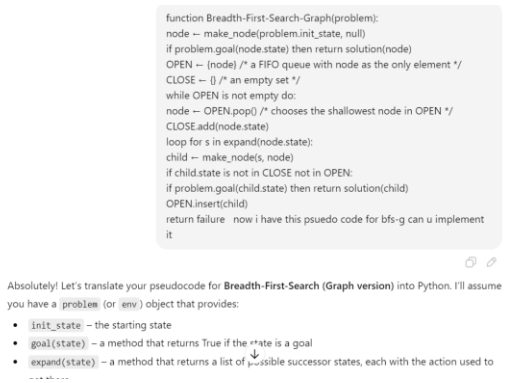
מוזמנים להסתכל במסמך המדיניות על דוגמאות לרמת הפירוט הנדרשת.
בנוסף, בחלק זה, אנו מזמינים אתכם לצרף כנספח את השיחה שלכם עם כלי ה-AI בנוגע לפתרון שיעורי הבית.

שימו לב! סטודנטים שרמת הפירוט שלהם בשאלה זו תהיה "לא משכנעת מספיק", ו/או שלא צירפו את השיחה שלהם עם כלי ה-AI איתו עבדו, יוכלו להיבחר בצורה מדגמית לבחינה קצרה בעל פה על הגיליון והקוד שהגישו.

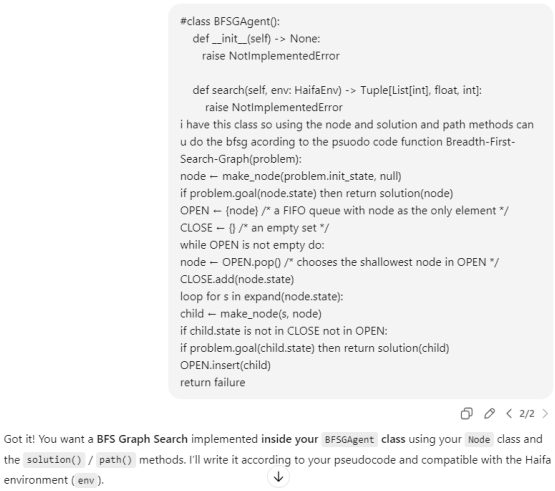
Wet part:
The environment: can u explain what s and a and s′ means?
The Haifa Environment : I gave gpt the HaifaEnv.py code and asked to tell me what′s in the code and what key points in it step by step. I also asked about the state vs. observation note
The random agent: I asked gpt to explain the code step by step.
Question 2 Bfs-G:
I asked gpt to help me with implementing the node and the solution classes



I asked gpt to implement the psuodo code of bfs-g

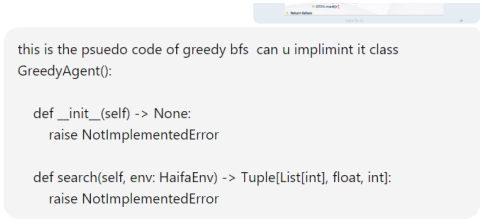


Now I asked him that using the node and solution methods we created can u implement the bfs-g pseudocode also using the init and search functions given in the algorithm.py file



Question 6:

I asked gpt to implement the pseudo code of the greedy agent



Below is a clean, correct, **fully working implementation** of **Greedy Best-First Search** that exactly follows your lecture's pseudo-code.

I assume the structure used in previous assignments:

And then using the bfs-g and greedy implementation guidelines I wrote gpt a pseudo code with how the env acts and also using the heapdict queue , and implementing the h_haifa heuristic .
the psudo code I gave gpt:

```
function Manhattan(s, g):
   (r1, c1) =get raw col from env
   (r2, c2) = get raw col from env
   return the manhatten dist between s and g

function h_HAIFA(state):
   goals = all goal states
   distances = [Manhattan(state, g) for g in goals]
   return min(min(distances), C_passway)



function Greedy_BFS(env):

   start = get_initial_state
   goals = get_goal_states

   OPEN = open a priority queue using heapdict such that the key is the h_haifa heuristic
        and if  two nodes have the same minimum value, your implementation should select the node with
        the lower state index first. Instead of defining priority as an integer, you can define it as a
        tuple (value, state, ...).
   CLOSED = empty set


push in open the start and its heuristic
   expanded = 0

   while OPEN is not empty:

      Pop the state with the lowest heuristic value (h, s)
              update the expande counter and add the node to closed list

      if s in goals:  # if we reached goal
         if the s state is the goal
            follow parent pointers backword to build the path
            and then compute the total cost of the path



         return path, total_cost, expanded

      # expand successors
      for each (ns, cost) in env.succ(s):
         if the the state expanded hit a wall or a hall (ns is none) continue
         if the state expanded is already in closed set continue
         if not in open then add it
```

return

question 7 :
I wrote a rough pseudocode for gpt and asked it to help me implement it and then tried a lot of different things with the code he gave me I tried to add an inserted order variable , and I tried to make the focal list not be a heapdict as written in the piazza , I also tried to not put the case 3 (the closed case) because the heuristic is consistent but that didn't work so I just kept it (all of that with the help of gpt)
Here is the pseudo code I gave to gbt
1. Initialize:

get the start node from the env
get the goals nodes from the env
lets use a open heapdict with the key(state) , and the value(f , state)
lets use a focal heapdict with the key (state) and the value (h, f ,state)
lets use a set for the nodes that we make node_info
we also need a close set
and we need an expanded node counter

2. Create start node:
create the first node \ the start node and initialize everything for it

3. While OPEN is not empty:
a. get the min value from the open list with min f
b. Build FOCAL:
build focal using the rule of the algo A star epsilon
c. Choose node with min h_focal from focal and remove it from open

d. If chosen_state in closed continue
e. Add chosen_state to CLOSED
f. expanded += 1

g. If chosen_state in goals:
return total cost , path , expanded

4. Expand successors of chosen_state:

For each successor succ of chosen_state

If succ is None, skip it.

Compute g_new , f_new , h_focal_new

Check if succ already exists in the list of nodes :

Case 1: new node

create the new node with everything in it
add it to the list of nodes
add it to open

Case 2: node in OPEN and g_new < existing.g

update existing node

update open

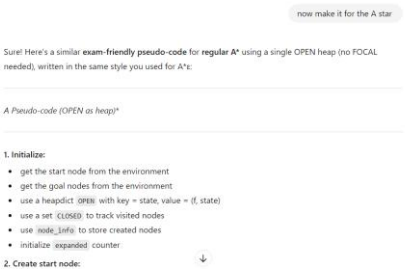
Case 3: node in CLOSED and g_new < existing.g

update existing node

add it back to open

remove it from closed

And for the A star code I updated the psudo code using gpt and then asked gpt to help implementing it something along these lines:



Dry part:

Question1:

this is the enviroment and this is my question and answer can u check if its right: קוד: עברו על המחברת עד שאתם מגיעים לחלק של BFS-G (לפני שתמשו את האלגוריתם הראשון במחברת) ועצרו שם.
שמופיעה במחברת (וכן 8"xהשאלות בחלק זה מתבססות על המפה "8 במסמך הנ"ל), אלא אם נכתב אחרת.
תחילה נרצה להגדיר את מרחב החיפוש כפי שנלמד בתרגול. נגדיר:
S=X^d = מרחב המצבים, אשר כל מצב בו מתואר על ידי וקטור של d פיצ'רים
O = מרחב האופרטורים
I = המצב ההתחלתי
G = קבוצת מצבי המטרה
עבור סביבת חיפה. (1 נק') הגדירו את (S,O,I,G)
State is defined as the state index on the map that is indexed from 0 to 63
S = { 0,1, .., 63}
Operators : O = { up ,down, left, right}
∀o∈O: Domain(o) = {s∈S }:
I = 0
G = 63

Question 2 :

I asked gpt to check if my answer is correct and to write it in better English

😊

2.2

2. מה צריך להיות התנאי על גרף החיפוש (לא בהכרח בבעיית הניווט בחיפה) כך שאלג' BFS (שרץ על עץ) ואלג' BFS-G (שרץ על גרף) ייצרו וייפתחו צמתים זהים באותו הסדר? (1 נק')
The condition is that 2 brothers at the same depth can't both point to the same successor because if they do first the first brother will expand the successor but also the second brother will expand it later which will make the the bfs tree expand the same succesor twice

Exactly — that's correct! 👍

To phrase it clearly for an answer:

Condition:

> For BFS on a tree and BFS-G on a graph to expand nodes in the same order, no two nodes at the same depth in the search tree may share the same successor.

Why:

- In BFS on a tree, each node is expanded independently, so the same successor could appear multiple times.

2.3

נתונה מפה בגודל NxN שלא מכילה Passways. הציעו דרך להשתמש באלגוריתם BFS-G כך שיחזיר פתרון אופטימלי (עלות מינימלית) והסבירו בקצרה. (2 נק')
רמז: מצאו פונקציה T:G→G המקבלת את גרף המצבים G ויוצרת גרף חדש G', ובעזרתה למצוא את המסלול האופטימלי בגרף G.
The idea is since Bfs gets the shortest path , and we have a weighted path , if an edge costs n>1 , we can replace it with n vertices that are connected with edges weighted by 1 (they are not connected to any other vertices).and then we can runc Bfs
can u check my answer

Yes — your answer is basically correct, but it can be polished a bit to make it clearer and more precise. Here's a checked and improved version:

Corrected version:

The idea is that BFS finds the **shortest path in terms of number of edges**, but in our map the edges have **weights (costs)**. To use BFS-G to find a **minimum-cost path**, we can transform the original weighted graph G into a new unweighted graph G' as follows:

- For each edge in G with cost $n > 1$, replace it with a **chain of n new intermediate vertices** connected

. 😊 After this I closed the ai tool and continued solving the dry part by myself, cause independent student of Technion.